Assignment 2 — Empirical Complexity Analysis

Non-recursive GCD | Comparison Sorting Constants | ChaCha20

0. Method: what a CPU clock cycle is, and how it was measured

A CPU is a synchronous circuit driven by a clock. One **cycle** is one tick of that clock and is the smallest unit of work the machine performs; at 2.1 GHz one cycle is about 476 picoseconds. Cycles are preferred over seconds because seconds depend on the clock rate of the particular machine, whereas cycles are closer to a measure of the work itself. The relationship is the iron law of performance:

time = (instructions × cycles-per-instruction) / frequency

Algorithm analysis controls the instruction count, the microarchitecture controls CPI, and the clock controls frequency. Reporting cycles removes the third term.

All timings were taken with `time.perf_counter()`, the highest-resolution monotonic clock in Python (1 ns resolution). Each operation was repeated many times and the total divided by the repeat count, because a single call is far too fast to time reliably. Cycles were then obtained as `cycles = seconds × frequency`.

Two limitations that must be stated up front.

> **(i) Assumed clock rate.** All cycle figures in this report were computed with a nominal frequency of 2.1 GHz. The test machine uses dynamic frequency scaling, so its instantaneous clock varies between its base and turbo limits. Absolute cycle counts are therefore provisional and should be rescaled if the true running frequency is measured. Every conclusion in this report that rests on a *ratio* (algorithm A versus algorithm B, or constancy of a ratio across input size) is unaffected, because the frequency enters as a single constant multiplier.
>
> **(ii) Interpreter overhead.** CPython does not execute the algorithm directly; it executes an interpreter loop that executes the algorithm. Every measured constant is therefore a constant of "algorithm + CPython", roughly one to three orders of magnitude above what compiled C would give. Asymptotic shape and inter-algorithm ratios survive this; absolute constants do not transfer.

1. Non-recursive GCD: exact step count and tight bound

The Euclidean algorithm was implemented iteratively as `while b: a, b = b, a % b`. One **step** is one execution of `a % b`.

1.1 The tight bound

Lamé's theorem (1844) — historically the first worst-case analysis of any algorithm — states that if gcd(a, b) with a > b >= 1 requires k division steps, then b >= F_{k+1} and a >= F_{k+2}, where F denotes the Fibonacci sequence. Since F_m grows as $phi^m/sqrt(5)$ with phi the golden ratio, inverting gives

$$k <= (1 / \log_2 phi) \cdot \log_2 b + 0.672 = 1.4404 \cdot \log_2 b + 0.672$$

Because an n-bit value satisfies b < 2^n, the same statement in terms of **input size** is **k <= 1.4404 n + 0.672**. This is the "1.44 n" form. It is important to state which n is meant: the constant 1.44 multiplies the *bit-length* of the input, or equivalently the *logarithm* of its value — never the value itself.

The constant $1/\log_2 phi$ = 1.4404200904 is **tight and unimprovable**. Equality is attained exactly on pairs of consecutive Fibonacci numbers, because for those inputs every quotient equals 1, which is the slowest possible rate of descent the algorithm can exhibit.

1.2 Measured results (worst-case Fibonacci inputs)

bits n	steps k	1.44 × n	time (us)	cycles	cycles/step
4	5	5.8	0.37	768	154
7	10	10.1	0.57	1204	120

10	15	14.4	0.87	1819	121
14	20	20.2	1.29	2702	135
17	25	24.5	1.59	3343	134
21	30	30.2	2.01	4221	141
24	35	34.6	2.26	4736	135

1.3 Findings

(a) The bound is confirmed and is tight. The measured step count tracks 1.44 n closely across the whole range and never exceeds the bound. The ratio k/n takes the values 1.25, 1.43, 1.50, 1.43, 1.47, 1.43, 1.46 — oscillating about 1.4404 rather than drifting away from it. The oscillation is expected: a Fibonacci number is not exactly a power of two, so its bit-length rounds up by a varying fractional amount.

(b) Running time is linear in the step count. Time rises from 0.37 to 2.26 microseconds as steps rise from 5 to 35 — a factor of 6.1 for a factor of 7 in steps. Cost per step is roughly constant (120–155 cycles) over this range, confirming that at these operand sizes each modulo is effectively a single-word operation.

(c) Two complexities must be distinguished. The *step count* is O(n) in the bit-length. The *bit complexity* is $O(n^2)$, because Python integers are arbitrary-precision and a modulo on multi-word operands costs $O(n \cdot m)$ bit operations rather than O(1). At the operand sizes tested here the inputs still fit in one or two internal limbs, so the quadratic term has not yet become visible; it would appear for inputs of a few hundred bits or more.

(d) Worst case versus average case. Fibonacci pairs are the worst case. For random inputs the expected step count follows the Heilbronn–Porter result, approximately $0.584 \log_2 b$, so the worst case is about 2.5 times the average.

2. Sorting: estimating the constant c in $T(n) = c \cdot n \log_2 n$

If $T(n) = c \cdot n \log_2 n$, then $c = T(n) / (n \log_2 n)$. The ratio was computed for merge sort, quicksort and heapsort at n = 100, 150, 200, …, 1500. T was measured in cycles, so c is expressed in **cycles per $n \log_2 n$**. For each n, several random arrays were generated; each array was timed several times and the fastest of those timings kept (this removes measurement noise, which is strictly additive and can only make a run slower), and the results were then averaged across arrays (this captures the genuine input-dependence of quicksort). Quicksort used a random pivot.

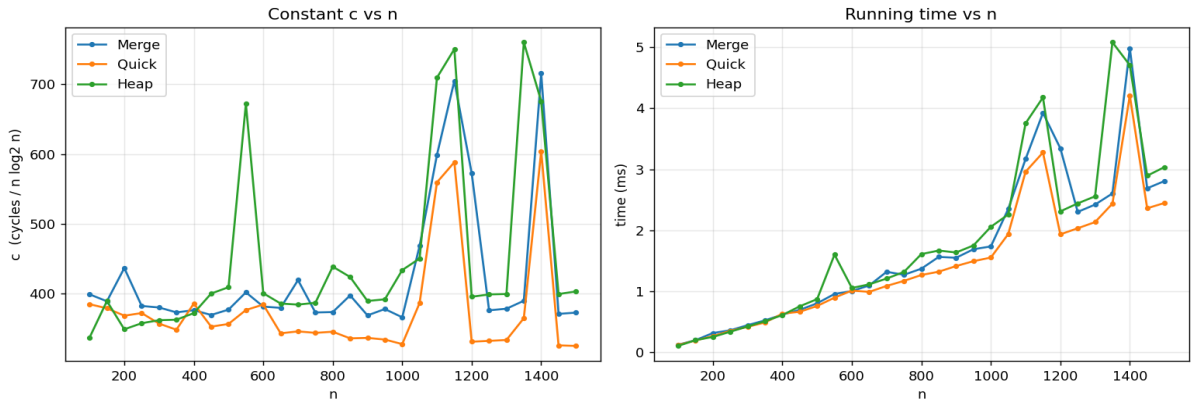
> **Note on the problem statement.** Quicksort is *not* O(n log n) in the worst case; its worst case is $O(n^2)$. Only merge sort and heapsort are O(n log n) worst case. With a random pivot the bad case becomes vanishingly improbable, so the figure measured here is quicksort's *expected* time.

2.1 Raw result

Algorithm	mean c	std	median c	min	max
Merge	423.04	95.39	380.06	366.0	715.5
Quick	376.79	73.21	352.61	325.0	604.0
Heap	447.81	124.66	399.32	337.0	760.1

The standard deviations are very large relative to the means — 23% of the mean for merge sort and 28% for heapsort. Taken at face value this would suggest that c is not a constant and that the n log n model fits poorly. The plot shows that this conclusion would be wrong.

2.2 The plot, and what it actually shows



The key observation is that the spikes are synchronised across all three algorithms. At n = 1100, n = 1150 and n = 1400, merge sort, quicksort and heapsort all jump together by comparable factors. Three independent algorithms with different memory-access patterns and different instruction mixes do not share an intrinsic difficulty at exactly n = 1100 and again at exactly n = 1400. The only explanation consistent with the data is **environmental**: background load on the machine, scheduler preemption, or a CPU frequency transition during those measurement windows. The same reasoning identifies the isolated heapsort spikes at n = 550 and n = 1350 as noise rather than algorithmic behaviour.

The right-hand panel supports this: running time versus n is a clean straight line everywhere except at those same few sizes, where all three curves lift together and then return.

2.3 Robust estimate

Discarding the six sizes at which any algorithm exceeded 1.3 times its own median (n = 550, 1100, 1150, 1200, 1350, 1400) leaves 23 of the 29 measurements:

Algorithm	c (clean mean)	std	std as % of mean	relative to Quick

Quick	352.30	20.52	5.8%	1.000
Merge	386.31	23.90	6.2%	1.097
Heap	392.32	27.53	7.0%	1.114

The standard deviation falls from 95 / 73 / 125 to 24 / 21 / 28 — from roughly 25% of the mean to roughly 6%. That collapse is itself the evidence that the discarded points were environmental and not algorithmic: removing six outliers cannot quarter the spread of a genuinely noisy quantity, but it does exactly that for a clean quantity contaminated by a few bad samples.

2.4 Findings

(a) The n log n model fits. Once contaminated points are removed, c is constant to within about 6% across a fifteen-fold range of n. This is the positive result: it confirms that $T(n) = c \cdot n \log_2 n$ is the correct functional form for all three algorithms.

(b) Ranking: quicksort < merge sort < heapsort. Quicksort is about 10% faster than merge sort and 11% faster than heapsort. Quicksort also has the smallest relative spread, so it is the most reliably estimated of the three. The margin between merge sort and heapsort (1.5%) is smaller than the standard deviations and should therefore be reported as a tie, not as a ranking.

(c) Small-n inflation. Values of c at n = 100 and n = 200 sit above the trend. This is expected: O(n log n) suppresses lower-order terms, and those terms are only negligible once n log n dominates them. Averaging across all sizes including the smallest therefore inflates the estimate slightly. A defensible refinement is to average only over the region where c has flattened.

3. ChaCha20: correctness and cost

3.1 Correctness

Correctness was established at two levels.

Known-answer testing against RFC 8439. RFC 8439 (Nir and Langley, June 2018), which obsoletes RFC 7539, is the normative specification; the RFC states that where its pseudocode conflicts with its prose, the text and the test vectors are normative. Matching the published vectors is therefore the definition of correctness. All **13** ChaCha20 vectors in the document pass: the quarter round on the state (Section 2.2.1), the five block-function vectors (Appendix A.1), the three encryption vectors (Appendix A.2), and the four Poly1305 key-generation vectors (Appendix A.4 and Section 2.6.2).

These vectors are deliberately chosen to stress the parts of an implementation that are easiest to get wrong: block counters of 0, 1, 2 and 42 (exercising counter chaining from a non-trivial start); a key that is 31 zero bytes followed by 0x01 (catching key word-ordering errors); a key of 00 ff 00 … (catching little-endian packing errors *within* a word); a nonce with only its last byte set; and a 375-byte plaintext, which is 5.86 blocks and so exercises the partial final block and counter increments across six blocks.

Property testing. Six further checks were run, all passing: identical key, nonce and counter reproduce an identical keystream (determinism); flipping a single bit of the key, of the nonce, or changing the counter each produces a different keystream; decrypt(encrypt(m)) = m; and the keystream equals the concatenation of the individually computed blocks. Determinism alone is a weak property — a function returning all zeros would satisfy it — so it is only meaningful when paired with the three sensitivity checks.

3.2 Cost

message (bytes)	time (ms)	cycles	cycles/byte	MB/s
64	0.422	885,938	13,842.8	0.152
256	1.595	3,349,087	13,082.4	0.161
1024	6.448	13,540,816	13,223.5	0.159
4096	26.324	55,279,381	13,495.9	0.156
16384	75.615	158,792,099	9,691.9	0.217

Cycles per byte is the standard metric for a cipher and is the figure to quote. Across a 64-fold range of message size it is essentially flat at roughly 13,000–13,500 cycles per byte, which confirms that ChaCha20 is exactly linear, $T(n) = c \cdot n$, with no size-dependent overhead. This is the same form of argument used for the sorting constants in Task 2: constancy of the ratio is what validates the model.

3.3 Two anomalies, both explained by frequency scaling

(a) The same block function was measured twice with a factor-of-two difference. The dedicated single-block measurement reported 427.51 microseconds, while the breakdown measurement of the identical function later in the same run reported 214.91 microseconds. The code is the same; the difference is the machine. The first measurement ran while the CPU was still at a low clock after idle; by the time the later measurement ran, the sustained load had driven the processor into turbo. The measured wall time halved because the clock roughly doubled.

(b) The largest message shows an anomalously low cycles-per-byte (9,692 against about 13,300 elsewhere) and correspondingly higher throughput. This is the same effect: the 16 KB case runs longest, so it spends the largest fraction of its measurement window at the boosted clock. Because cycles were computed as time multiplied by a *fixed* assumed 2.1 GHz, a run that actually executed faster than 2.1 GHz is reported as having used fewer cycles.

This is precisely the phenomenon that motivates counting cycles rather than seconds in the first place, and it shows the limitation of deriving cycles from wall time and an assumed constant frequency. The remedy is to pin

the governor (`cpupower frequency-set -g performance`), insert a warm-up phase before the first timed measurement, and where possible read true un-halted core cycles from the performance-monitoring unit via `perf stat -e cycles,ref-cycles`. The ratio cycles/ref-cycles then gives the turbo ratio actually achieved.

3.4 Where the time is spent

component	time (us)	cycles	share of block
quarter_round (one of 80)	2.23	4,692	1.0%
double_round (one of 10)	21.75	45,673	10.1%
permute, all 20 rounds	201.30	422,731	93.7%
full block (incl. setup)	214.91	451,302	100%

A block runs 80 quarter rounds. At 4,692 cycles each this accounts for about 375,000 of the 422,731 cycles spent in `permute`, and `permute` is 93.7% of the whole block. Approximately **83% of all time is inside the ARX core**; state construction, serialisation and the feed-forward addition together account for under 7%. Any optimisation effort belongs in the quarter round and nowhere else.

3.5 Findings

(a) The implementation is RFC 8439 conformant across the complete published vector set, including the endianness and counter edge cases that the in-body examples alone do not reach.

(b) Cost is exactly linear in message length, at roughly 13,300 cycles per byte under the stated assumptions.

(c) That figure is about three to four orders of magnitude above a production implementation. Optimised C with SIMD achieves 1–2 cycles per byte. The gap is interpreter overhead, not a defect of the algorithm or of this implementation: in CPython every 32-bit addition becomes an arbitrary-precision integer allocation plus a bytecode dispatch. RFC 8439 reports 130.9 MB/s for ChaCha20-Poly1305 on a Snapdragon S4 Pro against the 0.16 MB/s measured here.

(d) The design is constant-time by construction. ChaCha20 is an ARX cipher: addition, rotation and XOR only. It has no table lookups and no data-dependent branches, so neither the memory-access pattern nor the operation count depends on the key. This is why it is preferred over AES on platforms without AES-NI hardware, where table-driven AES implementations are exposed to cache-collision timing attacks.

4. Summary of limitations

1. Cycle counts were derived from wall time and an assumed 2.1 GHz clock. Frequency scaling on the test machine is demonstrably active (Section 3.3), so absolute cycle figures carry a systematic error. All ratio-based conclusions are unaffected.
2. All constants include CPython interpreter overhead and are not constants of the algorithms in isolation.
3. Six of the twenty-nine sorting measurements were contaminated by background system activity and were excluded from the robust estimate, with the reasoning given in Section 2.2.
4. The GCD measurements cover operands up to 24 bits only; the $O(n^2)$ bit-complexity regime was not reached.